

Lab 4 VibeCoding — Chat History with Claude

Tug of War Repair Lab (56_tug_of_war) — SE Lab, PES University

9 October 2026

Student repo (fork with task-wise commits): https://github.com/mythri2506a/56_tug_of_war

AI tool used: Claude (claude.ai)

This document is the exported chat between the student and Claude used to complete Lab 4. Each **Student** prompt is followed by Claude's **response**. Full final code for all changed files is in the Appendix.

1. Understanding the lab

Student: *(Shared the Lab 4 handout PDF, a screenshot of the 56_tug_of_war repo, the full README and main.py)*
“READ EVERYTHING WHICH I HAVE GIVEN AND PASTED AND PLS GUIDE ON WHAT I HAVE TO DO FROM STARTING guide pls”

Claude: Gave an overall plan for the 60-minute lab:

- **Deliverables (Lab-4 folder):** before video, after video, updated code with a separate commit per task, and the chat history (PDF + share link).
- Noted that the README's “Controls” line (HIGHER/LOWER buttons) was copied from another lab; the real controls are alternating **A** and **D** to pull left and **R** to restart.
- **Steps:** (1) fork and clone, `pip install pygame, python main.py`; (2) record the “before” video showing the input freeze; (3) start one dedicated AI chat and paste the README plus all code files; (4) do Tasks 1–4 one at a time, committing after each; (5) record the “after” video; (6) export the chat; (7) push to the Lab repo — never raise a PR to SETAPESU26.
- Suggested prompts for each task and advised pasting exact error tracebacks to keep each task within 3–4 attempts.

2. Setup

Student: “ok first what i ahve to do ?”

Claude: Fork the repo (Fork → Create fork), clone the fork, then install pygame and run the game:

```
git clone https://github.com/<your-username>/56_tug_of_war.git
cd 56_tug_of_war
pip install pygame
python main.py
```

Student: *(pasted error)* fatal: unable to access 'https://github.com/<mythri2506a>/56_tug_of_war.git/': The requested URL returned error: 400

Claude: The angle brackets were only a placeholder and must not be typed. Correct command:

```
git clone https://github.com/mythri2506a/56_tug_of_war.git
cd 56_tug_of_war
```

Student: *(pasted error)* ModuleNotFoundError: No module named 'pygame'

Claude: Install pygame with the same Python that runs the game: `python -m pip install pygame`. If that fails on a very new Python version, use the drop-in replacement `pygame-ce`.

Student: *(pasted error)* ERROR: Failed to build 'pygame' when getting requirements to build wheel

Claude: Plain pygame has no prebuilt package for this Python version (3.14), so pip tried to build from source and failed. The pip upgrade notice is harmless. Install **pygame-ce** instead — the code still uses `import pygame` unchanged:

```
python -m pip install pygame-ce
```

```
python main.py
```

Student: (screenshot: game running, “COMPUTER WINS!” after input froze) “this i got”

Claude: The game runs and shows the bug. Record the **before** video with Win + Alt + R (Xbox Game Bar) while mashing A and D until input freezes, save it as `before.mp4`, then paste the three game files.

3. Task 1 — Fix the alternating input deadlock bug

Student: (pasted full code of `game/game_engine.py`, `game/player.py`, `game/rope.py`)

Claude — root cause: In `handle_event`, pressing A pulls, sets `last_key = A`, and sets `is_pull_locked = True`. The lock is only released on A’s KEYUP. When mashing fast, D is pressed before A is fully released, so D’s KEYDOWN arrives while locked and is discarded. When A is then released, the lock clears — but D’s press is already lost, and the next A press is rejected because A is still `last_key`. Every overlapping press makes this worse until the rope stops responding.

Fix: remove the lock entirely. Each physical press produces exactly one KEYDOWN (key repeat is off by default in pygame), so checking that the key differs from the last key is enough to enforce alternation, and no press is ever dropped because another key is still held.

```
def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
        return

    # Every fresh KEYDOWN of the *other* key counts as a pull.
    # No lock / KEYUP dependency, so overlapping or very fast
    # alternating presses can never freeze input.
    if event.type == pygame.KEYDOWN and event.key in (pygame.K_a, pygame.K_d):
        if event.key != self.last_key:
            self.rope.pull_left(1.0)
            self.last_key = event.key
```

Also removed the unused `self.is_pull_locked = False` lines in `__init__` and `reset()`.

Student: “tell in which each file what i have to change”

Claude: Only `game/game_engine.py` changes for Task 1. Gave the complete file to paste in, and listed the three differences (lock removed from `__init__`, new `handle_event`, lock removed from `reset()`).

Student: (screenshot of the game running) “how do i check what changed ?”

Claude: Test in-game by alternating A and D quickly — the flag should keep moving left and never freeze; pressing A, A, A should do nothing. To see the code change, run `git diff` (red - = removed, green + = added; press q to exit).

Student: “what do u mean by mash a d a d”

Claude: “Mash” means press keys again and again very fast: A → D → A → D... using two fingers. Each switch pulls the rope left.

Student: (screenshot: “PLAYER WINS!”) “THIS I GOT”

Claude: Task 1 is fixed — input no longer freezes and the player can win. Commit:

```
git add .
git commit -m "Task 1: Fix alternating input deadlock bug"
```

4. Task 2 — Dynamic AI panic surges

Claude: Once the player has pulled the flag past a threshold toward their green line, the computer “panics”: its pull interval shrinks and its pull strength grows, scaling smoothly with how close the player is to winning. A flashing “**COMPUTER PANIC!**” message shows while active. Added `update_panic()` to `game_engine.py`, called every frame from `update()`, and reset the panic state in `reset()`.

Student: “YA I RECORDED BEFORE CHANGING THE CODE NOW AGAIN WHEN THE PLAYER WINS SHOULD I RECORD LIKE PRESSING THE A AND D ALTERNATIVELY ?”

Claude: No — the after video is recorded once, at the end, showing all four tasks together.

Student: “it is coming like computer win i am not able to win i tried full hard”

Claude: The surge was too strong. Tuned three values in `__init__`:

```
self.panic_start = 0.6          # was 0.5 - panic starts later
self.panic_min_cooldown = 150   # was 130 - fewer panic pulls
self.panic_max_strength = 1.2   # was 1.35 - weaker panic pulls
```

The computer still clearly fights back near the end, but the player can now win. Commit:

```
git add .
git commit -m "Task 2: Add dynamic AI panic surges"
```

5. Task 3 — Rope tension and puller leaning animations

Student: “ok next what done till here”

Claude: Progress summary (setup, before video, Tasks 1-2 done), then Task 3, which changes three files:

- **rope.py:** added a `tension` value (0 = slack, 1 = tight) and `get_y_at(x)`. When slack the rope sags in the middle; when tight it vibrates with a sine wave and turns brighter. The flag follows the rope’s shape.
- **player.py:** added `facing`, a `lean` angle smoothed toward a target, and rotated drawing around the feet, plus an arm reaching to the rope.
- **game_engine.py:** added `update_animations()` — player mashing raises activity, which (plus panic) sets rope tension; the flag’s smoothed velocity gives “momentum”, which makes the winning side lean back and the losing side tip forward.

Student: (pasted error) `AttributeError: 'Rope' object has no attribute 'set_tension'`

Claude: `game/rope.py` still had the old code, but the new `game_engine.py` calls `set_tension()`. Replace the whole `rope.py` with the new version, save, and check `player.py` was replaced too.

Student: “ok done i won next what ?”

Claude: Task 3 works. Commit:

```
git add .
git commit -m "Task 3: Add rope tension and puller leaning animations"
```

6. Task 4 — Match timer and sudden death

Claude: Only `game_engine.py` changes:

- A live timer counts up at the top of the screen (`update_timer()` / `render_timer()`).
- At **45 seconds** with no winner, **Sudden Death** turns on: the timer turns red, “SUDDEN DEATH! x2 POWER” flashes, and `power_multiplier()` returns 2.0, doubling both the player’s and the computer’s pulls.
- The timer freezes on Game Over, and pressing **R** resets the timer and sudden-death state.

Testing: hold the flag near the centre with medium-speed tapping until 0:45, check the banner and faster movement, then press R and confirm the timer returns to 0:00.

```
git add .
git commit -m "Task 4: Add match timer and sudden death mode"
```

7. Final video, export and submission

Student: “done final video and chat help”

Claude: Push the fork (`git log --oneline` to see the 4 commits, then `git push origin main`). For the after video, play until about 0:38, start recording, and keep mashing so the clip shows: no freeze (Task 1), panic (Task 2), rope/lean animation (Task 3), sudden death at 0:45 (Task 4). Then export the chat, build the Lab-4 folder, and upload it to the personal Lab repo.

Student: “i am not abke to do ctrl+p what to do ?” / “i did not do the chat export part can i add it later ?”

Claude: Alternatives: browser : menu → Print → Save as PDF; open the share link in Chrome/Edge; or copy the chat into Word and save as PDF. The chat files can be uploaded into the Lab-4 folder later, before the deadline.

Student: The printed share page came out blank, so the student asked Claude to produce one document containing everything. This PDF is that document.

Final Lab-4 folder structure:

```
Lab-4/
|-- before.mp4
|-- after.mp4
|-- chat_history.pdf
|-- chat_link.txt
`-- tug_of_war/
    |-- main.py
    |-- README.md
    `-- game/
        |-- game_engine.py
        |-- player.py
        `-- rope.py
```

Appendix A — Final `game/game_engine.py`

```
import random
import pygame
from game.rope import Rope
from game.player import Puller

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.rope = Rope(width, height)
        self.player = Puller(90, height // 2, (50, 120, 220), "PLAYER (A/D)", facing=1)
        self.computer = Puller(width - 90, height // 2, (220, 80, 50), "COMPUTER", facing=-1)

        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"

        # Computer pulling (base values)
        self.base_pull_cooldown = 180 # ms between pulls when calm
        self.computer_pull_cooldown = self.base_pull_cooldown
        self.last_computer_pull = pygame.time.get_ticks()

        # Panic surge settings (Task 2)
        self.panic_start = 0.6
        self.panic_min_cooldown = 150
        self.panic_max_strength = 1.2
        self.panic_active = False
        self.panic_intensity = 0.0

        # Animation state (Task 3)
        self.player_activity = 0.0
        self.momentum = 0.0
        self.prev_marker_x = self.rope.marker_x

        # Match timer & sudden death (Task 4)
        self.sudden_death_time = 45 # seconds before sudden death starts
        self.match_start_time = pygame.time.get_ticks()
        self.elapsed_seconds = 0.0
        self.sudden_death = False
```

```

self.font_big = pygame.font.SysFont(None, 48)
self.font_small = pygame.font.SysFont(None, 26)
self.font_timer = pygame.font.SysFont(None, 34)

def power_multiplier(self):
    """Double pulling power for everyone during sudden death."""
    return 2.0 if self.sudden_death else 1.0

def handle_event(self, event):
    if self.game_state != "PLAYING":
        if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
            self.reset()
            return

    # Every fresh KEYDOWN of the *other* key counts as a pull.
    # No lock / KEYUP dependency, so overlapping or very fast
    # alternating presses can never freeze input.
    if event.type == pygame.KEYDOWN and event.key in (pygame.K_a, pygame.K_d):
        if event.key != self.last_key:
            self.ropes.pull_left(1.0 * self.power_multiplier())
            self.last_key = event.key
            self.player_activity += 0.25

def update_timer(self):
    """Track match time and switch on sudden death at the limit."""
    now = pygame.time.get_ticks()
    self.elapsed_seconds = (now - self.match_start_time) / 1000.0
    if not self.sudden_death and self.elapsed_seconds >= self.sudden_death_time:
        self.sudden_death = True

def update_panic(self):
    """Work out how scared the computer is based on how close the player is to winning."""
    center = self.width / 2
    distance_to_player_goal = center - self.ropes.left_win_x
    progress = (center - self.ropes.marker_x) / distance_to_player_goal
    progress = max(0.0, min(1.0, progress))

    if progress >= self.panic_start:
        self.panic_active = True
        self.panic_intensity = (progress - self.panic_start) / (1.0 - self.panic_start)
    else:
        self.panic_active = False
        self.panic_intensity = 0.0

    self.computer_pull_cooldown = int(
        self.base_pull_cooldown
        - (self.base_pull_cooldown - self.panic_min_cooldown) * self.panic_intensity
    )

def update_animations(self):
    """Rope tension and puller leaning based on struggle and momentum."""
    self.player_activity *= 0.95
    tension = self.player_activity
    if self.panic_active:
        tension += 0.3 + 0.4 * self.panic_intensity
    if self.sudden_death:
        tension += 0.3
    self.ropes.set_tension(min(1.0, tension))

    velocity = self.ropes.marker_x - self.prev_marker_x
    self.prev_marker_x = self.ropes.marker_x
    self.momentum = self.momentum * 0.9 + velocity * 0.1
    advantage = max(-1.0, min(1.0, -self.momentum / 2.0))

    self.player.update_lean(6 + 14 * advantage)
    self.computer.update_lean(6 - 14 * advantage)

def update(self):
    if self.game_state != "PLAYING":
        return

    self.update_timer()
    self.update_panic()

    now = pygame.time.get_ticks()
    if now - self.last_computer_pull >= self.computer_pull_cooldown:
        computer_variance = random.uniform(0.7, 1.2)

```

```

        if self.panic_active:
            surge = 1.1 + (self.panic_max_strength - 1.1) * self.panic_intensity
            computer_variance *= surge
            computer_variance *= self.power_multiplier()
            self.rope.pull_right(computer_variance)
            self.last_computer_pull = now

    self.update_animations()

    result = self.rope.check_winner()
    if result:
        self.winner = result
        self.game_state = "GAME_OVER"  # timer freezes here (update stops running)

def reset(self):
    self.rope.reset()
    self.player.reset()
    self.computer.reset()
    self.last_key = None
    self.winner = None
    self.game_state = "PLAYING"
    self.computer_pull_cooldown = self.base_pull_cooldown
    self.panic_active = False
    self.panic_intensity = 0.0
    self.player_activity = 0.0
    self.momentum = 0.0
    self.prev_marker_x = self.rope.marker_x
    self.match_start_time = pygame.time.get_ticks()
    self.elapsed_seconds = 0.0
    self.sudden_death = False
    self.last_computer_pull = pygame.time.get_ticks()

def render_timer(self, screen):
    total = int(self.elapsed_seconds)
    minutes, seconds = divmod(total, 60)
    timer_text = f"TIME {minutes}:{seconds:02d}"
    color = (255, 80, 80) if self.sudden_death else (240, 240, 240)
    timer_surf = self.font_timer.render(timer_text, True, color)
    screen.blit(timer_surf, (self.width // 2 - timer_surf.get_width() // 2, 10))

def render(self, screen):
    screen.fill((30, 32, 36))

    mud_rect = pygame.Rect(self.width // 2 - 120, self.height // 2 - 80, 240, 160)
    pygame.draw.rect(screen, (45, 38, 30), mud_rect, border_radius=12)

    self.rope.render(screen)
    self.player.render(screen, self.rope.get_y_at(self.player.x + 30))
    self.computer.render(screen, self.rope.get_y_at(self.computer.x - 30))

    self.render_timer(screen)

    inst_surf = self.font_small.render(
        "Alternate [A] and [D] keys rapidly to pull!", True, (210, 210, 210)
    )
    screen.blit(inst_surf, (self.width // 2 - inst_surf.get_width() // 2, 40))

    # Flashing sudden death banner
    if self.sudden_death and self.game_state == "PLAYING":
        if (pygame.time.get_ticks() // 300) % 2 == 0:
            sd_surf = self.font_big.render("SUDDEN DEATH! x2 POWER", True, (255, 200, 50))
            screen.blit(sd_surf, (self.width // 2 - sd_surf.get_width() // 2, 75))

    if self.panic_active and self.game_state == "PLAYING":
        if (pygame.time.get_ticks() // 200) % 2 == 0:
            panic_surf = self.font_big.render("COMPUTER PANIC!", True, (255, 70, 70))
            screen.blit(
                panic_surf,
                (self.width // 2 - panic_surf.get_width() // 2, self.height - 90)
            )

    if self.game_state == "GAME_OVER":
        overlay = pygame.Surface((self.width, self.height), pygame.SRCALPHA)
        overlay.fill((0, 0, 0, 180))
        screen.blit(overlay, (0, 0))

        win_text = f"{self.winner} WINS!"

```

```

color = (80, 220, 80) if self.winner == "PLAYER" else (240, 80, 80)
text_surf = self.font_big.render(win_text, True, color)
screen.blit(
    text_surf,
    (self.width // 2 - text_surf.get_width() // 2, self.height // 2 - 50)
)

restart_surf = self.font_small.render(
    "Press [R] to Play Again", True, (240, 240, 240)
)
screen.blit(
    restart_surf,
    (self.width // 2 - restart_surf.get_width() // 2, self.height // 2 + 10)
)

```

Appendix B — Final `game/player.py`

```

import math
import pygame

class Puller:
    """Represents a puller character anchor on either side of the rope."""

    def __init__(self, x, y, color, label, facing=1):
        self.x = x
        self.y = y
        self.color = color
        self.label = label
        self.facing = facing          # +1 = rope is to the right, -1 = rope is to the left
        self.lean = 0.0               # degrees; positive = leaning backward (away from rope)
        self.font = pygame.font.SysFont(None, 24)

    def update_lean(self, target_lean):
        """Smoothly move the lean angle toward the target."""
        self.lean += (target_lean - self.lean) * 0.15

    def reset(self):
        self.lean = 0.0

    def render(self, surface, hand_y=None):
        """Draw avatar (leaning around the feet) and label."""
        pivot_x, pivot_y = self.x, self.y + 35          # feet
        theta = math.radians(self.lean) * (-self.facing)
        cos_t, sin_t = math.cos(theta), math.sin(theta)

        def rot(rx, ry):
            return (pivot_x + rx * cos_t - ry * sin_t,
                    pivot_y + rx * sin_t + ry * cos_t)

        # Body (rotated rectangle)
        body = [rot(-20, 0), rot(20, 0), rot(20, -70), rot(-20, -70)]
        pygame.draw.polygon(surface, self.color, body)

        # Head
        head = rot(0, -85)
        pygame.draw.circle(surface, (240, 210, 180), (int(head[0]), int(head[1])), 16)

        # Arm reaching to the rope
        if hand_y is not None:
            shoulder = rot(self.facing * 12, -55)
            hand = (self.x + self.facing * 30, hand_y)
            pygame.draw.line(surface, (240, 210, 180), shoulder, hand, 6)
            pygame.draw.circle(surface, (240, 210, 180), (int(hand[0]), int(hand[1])), 5)

        # Name / control tag
        label_surf = self.font.render(self.label, True, (240, 240, 240))
        surface.blit(label_surf, (self.x - label_surf.get_width() // 2, self.y + 45))

```

Appendix C — Final `game/rope.py`

```

import math

```

```

import pygame

class Rope:
    def __init__(self, screen_width, screen_height):
        self.screen_width = screen_width
        self.screen_height = screen_height
        self.center_y = screen_height // 2
        self.marker_x = screen_width // 2

        self.left_win_x = 180
        self.right_win_x = screen_width - 180
        self.pull_step = 12

        # Visual tension: 0.0 = slack (sags), 1.0 = very tight (vibrates)
        self.tension = 0.0
        self.max_sag = 18
        self.vibration_amp = 5
        self.ropes_left = 60
        self.ropes_right = screen_width - 60

    def pull_left(self, strength=1.0):
        self.marker_x -= int(self.pull_step * strength)

    def pull_right(self, strength=1.0):
        self.marker_x += int(self.pull_step * strength)

    def set_tension(self, tension):
        self.tension = max(0.0, min(1.0, tension))

    def check_winner(self):
        if self.marker_x <= self.left_win_x:
            return "PLAYER"
        if self.marker_x >= self.right_win_x:
            return "COMPUTER"
        return None

    def reset(self):
        self.marker_x = float(self.screen_width // 2)
        self.velocity = 0.0
        self.tension = 0.0

    def get_y_at(self, x):
        """Rope height at a given x: sag when slack, vibration when tight."""
        t = (x - self.ropes_left) / (self.ropes_right - self.ropes_left)
        t = max(0.0, min(1.0, t))
        envelope = math.sin(math.pi * t) # 0 at ends, 1 in the middle
        sag = (1.0 - self.tension) * self.max_sag * envelope
        now = pygame.time.get_ticks()
        vibration = (self.tension * self.vibration_amp
                     * math.sin(t * math.pi * 12 + now * 0.05)) * envelope
        return self.center_y + sag + vibration

    def render(self, surface):
        # Rope colour gets brighter as it gets tighter
        base = (180, 140, 90)
        tight = (235, 200, 120)
        color = tuple(int(b + (tt - b) * self.tension) for b, tt in zip(base, tight))

        points = [(x, self.get_y_at(x))
                  for x in range(self.ropes_left, self.ropes_right + 1, 6)]
        pygame.draw.lines(surface, color, False, points, 8)

        pygame.draw.line(surface, (50, 200, 50),
                         (self.left_win_x, self.center_y - 40),
                         (self.left_win_x, self.center_y + 40), 4)
        pygame.draw.line(surface, (200, 50, 50),
                         (self.right_win_x, self.center_y - 40),
                         (self.right_win_x, self.center_y + 40), 4)
        pygame.draw.line(surface, (120, 120, 120),
                         (self.screen_width // 2, self.center_y - 20),
                         (self.screen_width // 2, self.center_y + 20), 2)

        # Flag follows the rope's current shape
        flag_y = int(self.get_y_at(self.marker_x))
        flag_rect = pygame.Rect(int(self.marker_x) - 12, flag_y - 24, 24, 48)
        pygame.draw.rect(surface, (230, 40, 40), flag_rect, border_radius=4)

```

```
pygame.draw.rect(surface, (255, 255, 255), flag_rect, width=2, border_radius=4)
```